

SDN-Scale: AVL vs Red-Black

Otimização de Roteamento e Análise de Trade-offs

Estrutura de Dados II | Professor: Ricardo Sekeff

1. Fundamentação Teórica: O Custo da Busca e do Balanceamento

No projeto de sistemas de alta disponibilidade, o tempo de resposta é ditado pela eficiência da busca em conjuntos massivos de dados. Em redes SDN, tabelas de fluxo utilizam o conceito de *Longest Prefix Match* ou busca por prioridade, onde o desempenho de uma Árvore Binária de Busca (BST) convencional é insuficiente devido à sua vulnerabilidade ao desbalanceamento.

1.1. Árvores AVL: O Rigor do Equilíbrio

A Árvore AVL (Adelson-Velsky e Landis) é uma estrutura estritamente balanceada. Sua invariante define que, para qualquer nó, a diferença de altura entre suas subárvores esquerda e direita (FB) deve satisfazer $|FB| \leq 1$.

- **Custo de Busca:** Garantido em $O(\log n)$, com altura máxima $h < 1.44 \log_2(n + 2)$.
- **Custo de Manutenção:** Exige rotações frequentes, sendo ideal para sistemas *read-intensive* (onde o volume de buscas supera drasticamente o de inserções/remoções).

1.2. Árvores Red-Black (RBT): O Equilíbrio Pragmático

Diferente da AVL, a Red-Black foca em um balanceamento "relaxado" baseado em coloração de nós. Suas 5 propriedades fundamentais garantem que nenhum caminho da raiz até uma folha seja mais do que duas vezes mais longo que qualquer outro.

- **Custo de Busca:** Também $O(\log n)$, mas com uma altura ligeiramente superior à da AVL ($h \leq 2 \log_2(n + 1)$).
- **Custo de Manutenção:** Realiza menos rotações em média, utilizando *recoloring* para resolver conflitos. É preferível em sistemas *write-intensive* ou com alta volatilidade de dados.

2. Cenário de Projeto: Roteamento de Borda em Tempo Real

Vocês devem atuar como engenheiros de infraestrutura otimizando um *Load Balancer*. O sistema atual sofre de latência excessiva quando novas regras de firewall são inseridas sequencialmente. O objetivo é comparar a implementação da **AVL** e da **Red-Black** para decidir qual motor de busca oferece a menor latência em escala de **nanossegundos (ns)**.

3. Atribuições do Grupo (Divisão de Funções)

Integrante 1: Lead Software Engineer (Estruturas)

Missão: Implementar as classes `AVL_Router_Tree` e `RedBlack_Router_Tree`.

Desafio: Garantir que ambas as árvores gerenciem objetos `PacketRule` (ID, IP Origem/Destino, Prioridade). Na RBT, deve-se implementar rigorosamente as propriedades de coloração e rotação no *insert* e *delete*.

Comando: Implementar os métodos de balanceamento específicos. A AVL deve manter $|FB| \leq 1$ e a RBT deve respeitar as 5 propriedades fundamentais da coloração.

Integrante 2: DevOps & SRE (Stress Test e Coleta)

Missão: Executar testes de carga idênticos e comparativos.

Desafio: Utilizar a mesma semente (*seed*) de dados para ambas as estruturas, coletando tempos de busca, inserção e deleção em **nanossegundos**.

Comando: Gerar gráficos comparativos (Volume de Dados x Tempo) para 100.000 entradas ordenadas. Simular a expiração de regras removendo 20% dos nós.

Integrante 3: QA & Analytics (Auditoria e Análise)

Missão: Validar a integridade das árvores e realizar a análise de *trade-offs*.

Desafio: Implementar verificadores de invariantes (altura da AVL e propriedades de cor da RBT) para garantir que a lógica do Integrante 1 está correta.

Comando: Quantificar o número total de rotações em cada árvore. Realizar o *Code Review* obrigatório e assinar a análise técnica comparativa.

4. Dinâmica de Integração e Auditoria

Essa dinâmica de integração deverá ser usada durante o processo de desenvolvimento no GITHUB.

- ⇒ **Code Review Obrigatório:** O Integrante 1 não pode finalizar o merge sem que o Integrante 3 valide a lógica de altura e cores via testes unitários.
- ⇒ **Desafio da Deleção:** O rebalanceamento na exclusão é mandatório em ambas as árvores. O Integrante 2 deve reportar falhas estruturais após a remoção dos 20% das regras.
- ⇒ **Post-Mortem:** Documento simulando resposta técnica à diretoria justificando a escolha da estrutura ideal para o sistema baseando-se nos dados coletados.

Na prática, o que cada etapa dessa significa:

1. **Code Review Obrigatório (A Barreira de Qualidade):** Em um ambiente profissional, o desenvolvedor (Integrante 1) nunca tem "poder absoluto" sobre o código principal. Assim, antes de considerar a árvore pronta, o Integrante 3 (QA) deve realizar uma revisão linha por linha. O objetivo será encontrar erros de lógica que o autor não percebeu. Por exemplo, o Integrante 1 pode achar que a rotação está certa, mas o Integrante 3 nota que um ponteiro ficou "solto" ou que o Fator de Equilíbrio foi calculado errado. **A revisão de pares é a forma mais barata e eficiente de evitar bugs em produção.**
2. **O Desafio da Deleção (O Teste de Estresse):** Muitos de vocês aprendem a inserir dados em árvores, mas poucos dominam a remoção, que é muito mais complexa matematicamente. Assim, o Integrante 2 (SRE) deve forçar o sistema a deletar 20% das regras (simulando regras que expiraram). O objetivo é verificar se o Integrante 1 implementou o rebalanceamento na exclusão. Na AVL e na Red-Black, deletar um nó pode causar um "efeito dominó" de desbalanceamento que sobe até a raiz. **Um sistema de software deve ser resiliente não apenas na entrada de dados, mas também na sua manutenção e limpeza.**
3. **Relatório Post-Mortem (Comunicação Executiva):** Este é um termo comum no Vale do Silício. Quando um sistema cai ou apresenta lentidão, a equipe escreve um documento explicando o que aconteceu e como foi resolvido. Assim, um documento curto e direto endereçado à "diretoria" (ou ao professor, neste caso) deve ser feito todas as vezes que isso ocorrer (esperamos não acontecer). O objetivo é traduzir termos técnicos (como "rotação dupla à esquerda") para termos de negócio ("redução de latência de 50ms para 2ns"). **Aprenda a justificar suas decisões técnicas para pessoas que não são programadoras, mas que decidem o orçamento do projeto.**

5. Modelo de Entrega (Artigo Técnico)

O relatório deve seguir o modelo da SBC disponibilizado no site da disciplina no iCEV Digital. Você pode escolher utilizar o modelo $\text{\LaTeX}$, o modelo do Microsoft Word ou o OpenOffice. A escolha é da equipe. O que **NÃO** é negociável: usar outro modelo que não seja o que está disponibilizado no site. A escolha de outro modelo diferente do mencionado acima acarretará em atribuição de nota 0 (ZERO) a todos os membros da equipe.

Sugestão mínima de tópicos para o trabalho (não limitado a apenas esses):

- Introdução e Fundamentação.
- Metodologia (Linguagem e Seed).
- Desenvolvimento (Lógica AVL/RBT).
- Resultados Comparativos (ns).
- Discussão: Custo de Rotação.
- Conclusão: Estrutura Escolhida.
- Referências Bibliográficas
- Anexos

A seção de anexos pode conter elementos que ajudem a compreender o relatório, como um código ou uma listagem de dados. Entretanto, esta não é uma seção obrigatória.

6. Rúbrica de Avaliação

Critério	Peso	Foco da Auditoria
Lógica AVL & RBT	4.0	Invariantes de balanceamento e tratamento de ponteiros.
Análise de Performance	3.0	Precisão em nanossegundos e uniformidade dos testes.
Gestão Git & Review	2.0	Divisão de tarefas comprovada e histórico de branches.
Qualidade do Artigo	1.0	Modelo da SBC (formatação), terminologia e análise crítica.

"Na engenharia, não existe solução perfeita, apenas a solução correta para o problema certo."